

1. Hafta

Web'in Temelleri: Ağdan HTTP'ye

© Öğrenme Hedefleri

Bu bölümü tamamladıңызda:

- TCP/IP katman modelini (uygulama, taşıma, İnternet, bağlantı), IP adresi ve port kavramlarını, DNS'in alan adı çözümlemedeki rolünü açıklayabilecek; YG II'de yazdığınız soket tabanlı istemci/sunucunun bu katmanlardaki yerini gösterebileceksiniz;
- Standart bir web sunucusunun mimarisini (istemci-sunucu modeli, statik ve dinamik içerik ayrımı, ters vekil kavramı, port 80/443 sözleşmesi) açıklayabileceksiniz;
- Modern bir WebApp'in üç katmanlı mimarisini (sunum, iş, veri), API kavramını ve SPA ile SSR arasındaki farkı ayırt edip bu dersin yığının (React, FastAPI, SQLite) bu şemaya yerleştirebileceksiniz;
- Bir HTTP isteğinin ve cevabının anatomisini (metot, yol, başlıklar, gövde, durum kodu) `curl` ve tarayıcı DevTools ile inceleyebileceksiniz;
- `curl` ile gerçek bir sunucuya istek gönderip ham HTTP değişimini okuyarak, öğrendiğiniz katmanların (ağ → sunucu mimarisi → WebApp mimarisi → HTTP) Etkinlik/Bilet Sisteminin mimari haritasında nasıl bir araya geldiğini yorumlayabileceksiniz.

Bu ders boyunca bir Etkinlik/Bilet Sistemi inşa edeceksiniz: kullanıcıların etkinlikleri görüp koltuk seçip bilet aldığı, tarayıcıda çalışan bir web uygulaması. Ama kod yazmaya başlamadan önce bir soruyu yanıtlamanız gerekiyor: tarayıcınıza bir adres yazdığınızda, o adresin arkasında gerçekte ne oluyor? YG I/II'de C/C++, Qt ve SQLite ile yazdığınız uygulamalar hep tek bir bilgisayarda, tek bir süreç içinde çalıştı; web uygulamaları ise doğası gereği *ağ üzerinden* konuşan, birbirinden ayrı bileşenlerden oluşur. Bu hafta, hiç kod yazmadan, bu ayrışmanın dört katmanını sırayla açacaksınız: en alttaki fiziksel ağdan başlayıp yukarı doğru, bir web sunucusunun standart mimarisine, oradan modern bir WebApp'in kendi iç katmanlarına ve son olarak tüm bunların ortak dili olan HTTP'nin isteğe/cevaba dönüşen somut anatomisine ulaşacaksınız. YG II'nin soket haftasında `connectToHost(ip, port)` ile kurduğunuz bağlantının üstüne, şimdi asıl web'in konuştuğu protokolü bindireceksiniz.

1.1 TCP/IP Katmanları, IP Adresi, Port ve DNS

1 • Ön-öğretim 📢 Neden Önemli?

YG II’de bir `QTcpSocket` ile `connectToHost("127.0.0.1", 5000)` yazdığınızda IP adresi ve port sizin için zaten hazır veriliyordu — siz bunları elle, bir dizge ve bir tam sayı olarak koda koydunuz. Peki tarayıcınıza yalnızca `www.google.com` yazdığınızda ne oluyor — bu isim nereden bir IP adresine, bir de porta dönüşüyor? Bu ünite, adres çubuğuna yazdığınız bir kelimenin arkasında dönen, görünmez ama her web isteğinde tekrar eden bu süreci açar.

1 • Ön-öğretim 🔄 Ön Bilgi

Bu üniteye girmeden önce YG II’nin TCP soket haftasından şunları hatırlayın: `QTcpSocket/QTcpServer` ile yazdığınız istemci/ sunucu çifti, `connectToHost(ip, port)` çağrısı, sunucu tarafında `bind/listen` kavramları. Bu ünite bu deneyime iki şey ekler: IP adresinin ve portun *ne olduğunu* isimlendirmek, ve DNS’in bu ikiliyi bir alan adından nasıl bulduğunu göstermek.

Bir web sayfası istediğinizde, tarayıcınızın attığı ilk adım aslında ağla hiç ilgili değildir: hangi makineye, hangi kapıdan gireceğini bulmaktır. Bu bölüm, bu bulma işleminin altındaki katmanlı yapıyı ve bu yapının en üstünde duran DNS’i sırasıyla açar.

1.1.1 Basitleştirilmiş dört katmanlı model

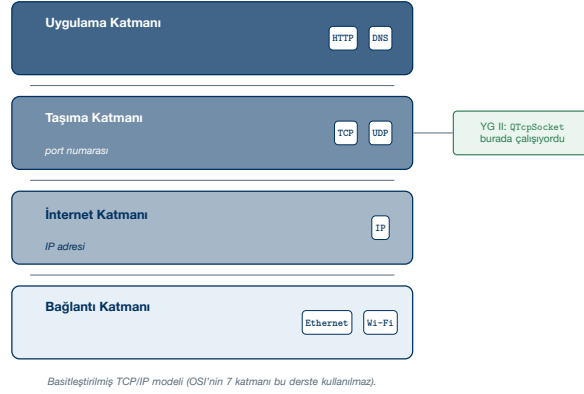
İnternet üzerinden giden her bayt, aslında üst üste binmiş birkaç ayrı sorumluluk katmanından geçer. Gerçek dünyada bu katmanlar (OSI modelinde) yediye kadar çıkabilir, ama günlük web programlama işinizde bu kadar ayrıntıya ihtiyacınız yoktur. Bu derste, İnternetin fiilen üzerinde çalıştığı, sadeleştirilmiş **TCP/IP katman modeli** ile yetineceksiniz.

Dört katmanı en üstten en alta doğru sırayla düşünün:

- **Uygulama katmanı:** sizin doğrudan kullandığınız protokoller burada yaşar — HTTP (web sayfaları), DNS (isim çözümleme). Bu ders boyunca en çok bu katmanla, özellikle HTTP’yle ilgileneceksiniz.
- **Taşıma katmanı:** verinin *güvenilir* mi yoksa *hızlı ama garantisiz* mi gideceğine karar veren katman — TCP (sıralı, kayıpsız teslim garantisi verir) ve UDP (garantisiz ama düşük gecikmeli). Bu ders TCP’ye odaklanır, çünkü HTTP TCP üzerinde çalışır. Port numarası da bu katmanın işidir.
- **İnternet katmanı:** verinin hangi *makineye* gideceğini belirleyen katman — IP protokolü, adresleme ve yönlendirmeden sorumludur.
- **Bağlantı katmanı:** verinin fiziksel olarak nasıl taşındığı — Ethernet kablosu, Wi-Fi sinyali. Bu ders bu katmana hiç dokunmaz, ama var olduğunu bilmeniz yeterlidir.

Şekil 1.1 bu dört katmanı, her birinin tipik protokolleriyle birlikte gösterir; taşıma katmanı hizasındaki not, YG II’de `QTcpSocket` ile çalıştığınız yerin tam olarak bu katman olduğunu işaretler.

TCP/IP katman modeli:
ağ iletişimini dört sorumluluk katmanına (uygulama, taşıma, İnternet, bağlantı) ayıran basit model (*TCP/IP layer model*).



Şekil 1.1: Basitleştirilmiş dört katmanlı TCP/IP modeli: uygulama, taşıma, İnternet, bağlantı katmanları ve her birinin örnek protokolleri.

Bu dört katmanı ayrı ayrı düşünmenin pratik faydası şudur: bir web isteği başarısız olduğunda, sorunun *hangi katmanda* olduğunu sorarak arama alanınızı daraltırsınız. Alan adı hiç çözümlenmiyorsa sorun muhtemelen DNS'te (uygulama katmanı); bağlantı zaman aşımına uğruyorsa sorun muhtemelen İnternet/bağlantı katmanında (makineye hiç ulaşılamıyor); bağlantı kuruluyor ama sunucu hatalı bir cevap veriyorsa sorun muhtemelen uygulama katmanındadır (HTTP düzeyinde). Bu ayırım, gelecekte karşılaştığınız ağ sorunlarını debug ederken size bir kontrol listesi verir.

1.1.2 IP adresi: doğru makineyi bulmak

IP adresi: bir ağ arayüzünü ağ üzerinde tekil olarak tanımlayan sayısal adres (*IP address*).

IP adresi, İnternete bağlı her makineyi (ya da makinedeki bir ağ arayüzünü) diğerlerinden ayıran sayısal bir kimliktir. IPv4'te bu adres dört sayının nokta ile ayrılmasıyla yazılır, ör. 93.184.216.34. YG II'de `connectToHost("127.0.0.1", 5000)` yazdığınızda kullandığınız 127.0.0.1 de bir IP adresiydi — “bu makinenin kendisi” anlamına gelen özel bir adres (*loopback*).

IP adresini bir postane adresine benzetebilirsiniz: bir mektubun size ulaşması için önce doğru *binaya* gitmesi gerekir; IP adresi de verinin ağ üzerinde doğru *makineye* gitmesini sağlar. Ama bir binada onlarca daire olabilir — bina adresi tek başına hangi daireye gideceğinizi söylemez. İşte bu, port'un işidir; bir sonraki alt bölümde bunu açacaksınız.

1.1.3 Port: aynı makinedeki doğru servisi bulmak

Tek bir makine (tek bir IP adresi) aynı anda birden çok servis çalıştırabilir: bir web sunucusu, bir e-posta sunucusu, bir SSH sunucusu — hepsi aynı IP adresinde, ama farklı “kapılardan” dinlerler. Bu kapıyı isimlendiren sayıya **port** denir.

port: aynı IP adresi üzerindeki servisleri birbirinden ayıran 0–65535 arası sayı (*port*).

Analoji

IP adresi ile portu birlikte bir zarf üstündeki adrese benzetin: posta kodu, cadde ve bina numarası (IP adresi) mektubu doğru *binaya* götürür; daire numarası (port) ise aynı binadaki doğru *kapıya* götürür. Aynı binada onlarca daire olabilir — her biri farklı bir kişiyi, farklı bir hizmeti temsil eder. Postacı binaya kadar gelir ama daire numarasını bilmeden hangi kapıyı çalacağını bilemez; tıpkı bir paketin doğru makineye ulaşip da doğru port numarası olmadan hangi servise

teslim edileceğinin belirsiz kalması gibi.

Bu benzetmeyi doğrudan YG II'deki kodunuza eşleyin: `QTcpSocket`'in `connectToHost(ip, port)` çağrısındaki ilk argüman (`ip`) sizi doğru *binaya*, ikinci argüman (`port`) doğru *kapıya* götürüyordu. Bu ünite, o çağrının iki argümanının aslında ne anlama geldiğini isimlendirmiş oldu.

Port numaraları 0'dan 65535'e kadar gidebilir, ama bunların bir kısmı "iyi bilinen" (*well-known*) portlardır — belirli servisler için evrensel olarak ayrılmış numaralardır: 80 (HTTP), 443 (HTTPS), 22 (SSH). Bu üç port numarasını şimdilik akılda tutmanız yeterli; port 80/443'ün web için tam olarak neden bir "sözleşme" olduğunu Ünite 2'de derinlemesine göreceksiniz.

Ortak Hata / Tuzak

Yeni başlayanların sık düştüğü bir karışıklık: "IP adresi bir makineyi, port da bir *programı* mı belirtir?" sorusuna evet demek yarı doğrudur ama eksiktir. Port, bir programı değil, o makinede *o anda dinleyen bir servisi* belirtir — aynı program (ör. bir web sunucusu yazılımı) aynı anda birden fazla porttan dinleyebilir, ya da bir port zaman içinde farklı programlar tarafından kullanılabilir. Port, programın kimliği değil, o anki *dinleme noktasının* kimliğidir.

1.1.4 DNS: isimden adrese

Gerçekte hiç kimse `93.184.216.34` gibi sayıları ezberleyip tarayıcıya yazmaz; `www.example.com` yazarsınız. Bu ismi IP adresine çeviren sistemin adı **DNS**dir.

DNS:

insan tarafından
hatırlanabilir alan
adlarını

(`www.example.com`) IP
adresine çeviren dağıtık
isim çözümleme sistemi
(*Domain Name
System*).

Analoji

DNS'i 118 numaralı bilgi/rehber santraline benzetebilirsiniz: bir kişinin adını bildiğinizde ama telefon numarasını bilmediğinizde santrali arar, adı söylersiniz; santral size numarayı verir. Tarayıcı da `www.example.com` adını bildiğinde, gerçek IP adresini bilmediği için DNS'e sorar; DNS bu ismi karşılık gelen IP adresine çevirir ve tarayıcı artık bu IP adresine, uygun portta bağlanabilir.

Çözümleme adımlarını basitleştirilmiş olarak şöyle düşünebilirsiniz: tarayıcı önce kendi **yerel ön belleğine** bakar (bu isim daha önce çözümlendiyse tekrar sormaya gerek yoktur); yoksa işletim sisteminin kullandığı bir **çözümleyiciye** (*resolver*, genelde İnternet servis sağlayıcınızın ya da işletim sisteminizin yapılandırdığı bir sunucu) sorar; çözümleyici de gerekirse alan adı için **yetkili sunucuya** (*authoritative server*) danışıp cevabı tarayıcıya döndürür. Kök/TLD/yetkili sunucu hiyerarşisinin iç mimarisine bu haftanın derinliğinde girmenize gerek yok; önemli olan, bir ismin bir adrese *dağıtık* bir sorgulama zinciriyle dönüştüğünü kavramanızdır.

İpucu

DNS çözümlemesinin sonucu genelde bir süre ön belleğe alınır (bir "time-to-live" süresi boyunca). Bu yüzden bir alan adının IP adresini değiştirdiğinizde (ör. sunucuyu taşıdığınızda) değişikliğin tüm dünyada görünür olması anında değil, ön bellek sürelerine bağlı olarak dakikalar ya da saatler alabilir.

1.1.5 Canlı gözlem: dig ve curl -v

Bu iki kavramı (DNS çözümleme ve port) komut satırında canlı olarak gözlemleyebilirsiniz. Aşağıdaki kesit, **dig** komutuyla bir alan adının hangi IP adresine çözümendiğini gösterir (Windows'ta karşılığı **nslookup**'tur):

```
1 if ! command -v dig >/dev/null 2>&1; then
2     echo "HATA: 'dig' komutu bulunamadi. Windows'ta yerine 'nslookup ${
3         ALAN_ADI}' calistirin." >&2
4 else
5     DIG_CIKTI=$(dig +short "${ALAN_ADI}" A 2>&1)
6     DIG_DURUM=$?
7     if [ ${DIG_DURUM} -ne 0 ] || [ -z "${DIG_CIKTI}" ]; then
8         echo "HATA: '${ALAN_ADI}' cozumlenemedi. Internet/DNS baglantinizi
9             kontrol edin." >&2
10    else
11        echo "${DIG_CIKTI}"
12    fi
13 fi
```

dig +short yalnızca çözümlenen IP adres(ler)ini basar — bu, tarayıcının DNS'e sorduğunda arka planda aldığı cevabın ta kendisidir. Şimdi aynı alan adına gerçek bir bağlantı kurulduğunda hangi porta gidildiğini görün:

```
1 CURL_CIKTI=$(curl -sS -v -m 10 "https://${ALAN_ADI}" -o /dev/null 2>&1)
2 CURL_DURUM=$?
3 if [ ${CURL_DURUM} -ne 0 ]; then
4     echo "HATA: https://${ALAN_ADI} adresine baglanilamadi (curl kodu: ${
5         CURL_DURUM}). Internet baglantinizi kontrol edin." >&2
6 else
7     echo "${CURL_CIKTI}" | grep -i "Connected to"
8 fi
```

curl -v çıktısındaki **Connected to ... port ...** satırı, DNS'in bulduğu IP adresine, hangi porttan bağlanıldığını gösterir — **https://** kullanıldığı için burada port **443** görünür (HTTPS'in varsayılan portu; Ünite 2'de bu sözleşmeyi derinlemesine göreceksiniz).

3 • Katılım Şimdi Siz Deneyin

Bilgisayarınızda **dig www.example.com** (Windows'ta **nslookup www.example.com**) çalıştırın. Çıktıda hangi satır IP adresidir? Ardından **curl -v https://www.example.com** çalıştırıp **Connected to** ile başlayan satırı bulun — bu satırda gördüğünüz port numarası kaçtır ve neden?

Geribildirim. **dig** çıktısında **ANSWER SECTION** altındaki satır (ya da **nslookup** çıktısında **Address:** ile başlayan satır) IP adresidir. **curl -v** çıktısında **Connected to www.example.com (IP) port 443** benzeri bir satır görürsünüz — port **443**'tür, çünkü **https://** önekini kullandınız ve HTTPS'in varsayılan sözleşme portu **443**'tür (Ünite 2'nin konusu).

Kendinizi Sınavın 1.1

Bir web sitesine tarayıcıdan eriştiğinizde IP adresi ile port numarasının birlikte hangi işi yaptığını, DNS'in bu ikiliyi bulmanıza nasıl yardım ettiğini açıklayın.

5 • İzleme ➔ Aktarım — Başka Nerede?

Bu hafta öğrendiğiniz IP+port ikilisi, Hf.6'da FastAPI sunucunuzu `uvicorn app.main:app --port 8000` ile ayağa kaldırdığınızda somutlaşacak — `localhost:8000` yazdığınızda tarayıcınız yine bu ünitenin öğrettiği IP+port çözümlemesini (bu kez kendi yerel makinenizde, DNS'e hiç gitmeden) yapacak. `localhost` özel bir isimdir ve doğrudan `127.0.0.1`'e karşılık gelir — bu yüzden Hf.6'da DNS'e ihtiyacınız olmayacak, ama IP+port ikilisinin mantığı birebir aynı kalacak.

1.2 Standart Web Sunucu Mimarisi

1 • Ön-öğretim ➔ Neden Önemli?

Aynı web sitesine giren iki farklı kullanıcı bazen birebir aynı sayfayı (ör. şirketin “Hakkımızda” sayfası) görür, bazen tamamen farklı bir sayfa görür (ör. kendi “hesabım” sayfası). Ayrıca büyük siteler tek bir sunucu makinesiyle çalışmaz; gelen milyonlarca isteği bir şekilde paylaşırlar. Bu ünite, bir web sunucusunun bu ikisini — sabit/ değişken içerik ayrımını ve trafik yönetimini — nasıl standart bir mimariyle çözdüğünü açar.

1 • Ön-öğretim ➔ Ön Bilgi

Ünite 1'in istemci/sunucu, IP/port kavramlarını hatırlayın. Ayrıca YG II'de yazdığınız TCP sunucusunun genelde **tek bir bağlantıyı** (ya da `std::thread` ile birkaçını) dinlemesi ile gerçek bir web sunucusunun (nginx gibi) aynı anda binlerce bağlantıyı yönetmesi arasındaki ölçek farkını aklınızda tutun — bu ünite bu farkın nasıl kapatıldığını gösterir.

Ünite 1, bir isteğin doğru makineye ve doğru porta nasıl ulaştığını gösterdi. Bu ünite, o kapıdan giren isteği bir web sunucusunun fiilen nasıl karşıladığını, hangi rolleri paylaştığını açar.

1.2.1 İstemci–sunucu modeli

Web'in temel iletişim biçimi, rollerin net biçimde ayrıldığı **istemci–sunucu modelidir**. Tarayıcınız (ya da `curl`) **istemcidir**: isteği her zaman o başlatır. Sunucu ise **bekler**: bir istek geldiğinde cevap verir, ama kendiliğinden hiçbir zaman istemciye bir sayfa göndermez.

istemci–sunucu modeli:
isteği başlatan tarafın (istemci) ile cevaplayan tarafın (sunucu) rollerine ayrıldığı iletişim modeli (*client–server model*).

➔ Analoji

Bunu bir gişeye ile müşteri ilişkisine benzetin: gişedeki görevli (sunucu) her zaman orada bekler, kimseyi aramaz; müşteri (istemci) gişeye gidip talebini iletir, görevli o talebe göre cevap verir. Görevli kendiliğinden sokağa çıkıp müşteri aramaz — tıpkı bir web sunucusunun, siz bir istek göndermeden size hiçbir şey göndermemesi gibi.

Bu ayrım, YG II'nin soket haftasındaki istemci/sunucu rolleriyle birebir aynıdır — şimdi yalnızca bu genel modelin HTTP'ye özelleştiğini göreceksiniz.

QTcpServer'ın `listen()` ile beklemesi ile bir web sunucusunun 80/443 portunda beklemesi aynı fikrin iki farklı ölçekteki uygulamasıdır.

1.2.2 Statik içerik: her istekte aynı cevap

statik içerik: sunucunun diskte hazır duran bir dosyayı (HTML, CSS, resim) olduğu gibi gönderdiği durum (*static content*).

Bir sunucunun verebileceği en basit cevap türü, **statik içerik**dir. Sunucu, isteği aldığı anda diskte zaten var olan bir dosyayı bulur ve olduğu gibi gönderir — hiçbir hesaplama yapmaz, sadece bir dosyayı kopyalayıp gönderir. Aynı URL'e kaç kez istek atarsanız atın, kim atarsa atsın, cevap birebir aynıdır (dosya değişmediği sürece).

1.2.3 Dinamik içerik: her istekte yeniden üretilen cevap

dinamik içerik: sunucunun her istekte isteğe/ duruma göre anında ürettiği içerik (*dynamic content*).

Buna karşılık **dinamik içerik**, sabit bir dosya değil, sunucunun o anki isteğe göre **hesapladığı** bir cevaptır. Örnek: kişiselleştirilmiş bir "hesabım" sayfası (kim istiyorsa ona göre farklı), ya da güncel etkinlik listesi (zaman geçtikçe değişebilir).

Analoji

Statik ile dinamik içeriği bir duvardaki afiş ile gişe memurunun sözlü cevabına benzetin: duvardaki afiş (statik) kim baksa aynıdır, sabittir, değişmez. Gişe memuruna "bugün kaç bilet kaldı?" diye sorduğunuzda (dinamik) ise cevap her an değişebilir — memur soruyu her seferinde yeniden yanıtlar, önceden basılmış bir kâğıt göstermez.

Bu iki cevap türünü doğrudan karşılaştırarak görün. Aşağıdaki kesit aynı statik bir sayfaya iki kez, aynı dinamik bir uç noktaya iki kez istek atıp cevapları karşılaştırır:

```
1 ISTEK1=$(curl -sS -m 10 "${STATIK_URL}")
2 DURUM1=$?
3 ISTEK2=$(curl -sS -m 10 "${STATIK_URL}")
4 DURUM2=$?
5
6 if [ ${DURUM1} -ne 0 ] || [ ${DURUM2} -ne 0 ]; then
7     echo "HATA: ${STATIK_URL} adresine baglanilamadi. Internet baglantinizi
8     kontrol edin." >&2
9 elif [ "${ISTEK1}" = "${ISTEK2}" ]; then
10    echo "Statik cevap iki istekte de BIREBIR AYNI (beklenen davranis)."

```

```
1 CEVAP1=$(curl -sS -m 10 "${DINAMIK_URL}")
2 DURUM3=$?
3 CEVAP2=$(curl -sS -m 10 "${DINAMIK_URL}")
4 DURUM4=$?
5
6 if [ ${DURUM3} -ne 0 ] || [ ${DURUM4} -ne 0 ]; then
7     echo "HATA: ${DINAMIK_URL} adresine baglanilamadi. Internet baglantinizi
8     kontrol edin." >&2
9 else
10    echo "1. cevap: ${CEVAP1}"
11    echo "2. cevap: ${CEVAP2}"
12    if [ "${CEVAP1}" != "${CEVAP2}" ]; then
13        echo "Dinamik cevap iki istekte FARKLI (beklenen davranis)."
```

```

13 else
14     echo "Dinamik cevap iki istekte aynı çıktı (beklenmiyordu – sunucu
        onbellek olabilir)."
```

```

15 fi
16 fi
```

example.com'un ana sayfası sunucudaki hazır bir HTML dosyasını olduğu gibi gönderdiği için iki ardışık istek birebir aynı gövdeyi döndürür; httpbin.org/uuid ise her istekte **yeniden üretilen** bir rastgele kimlik döndürdüğü için iki ardışık istek farklı bir gövdeyle sonuçlanır. Sunucunun bir isteğe statik mi dinamik mi cevap vereceği, çoğu zaman isteğin *yoluna* bakılarak belirlenir — ama bu ayrımın kod düzeyinde nasıl kurulduğunu Hf.6'da FastAPI ile göreceksiniz.

Ortak Hata / Tuzak

“Dinamik” kelimesi bazen yanlış anlaşılır: dinamik içerik her zaman *rastgele* ya da *gerçek zamanlı* anlamına gelmez, yalnızca sunucunun cevabı önceden hazırlamayıp **isteğe göre hesapladığı** anlamına gelir. Bir dinamik uç nokta bile bazen aynı sonucu verebilir (ör. veritabanında hiçbir şey değişmediyse); ayırt edici olan cevabın *hazır bir dosyadan mı yoksa o anki hesaplamadan mı* geldiğidir.

1.2.4 Port 80/443 sözleşmesi

HTTP varsayılan olarak **80** portunda, HTTPS ise **443** portunda dinler. Bunun önemli bir yanı var: bu bir teknik zorunluluk değil, evrensel bir **sözleşmedir** — tarayıcı, URL'de açıkça bir port belirtilmediyse bu varsayılanı dener (http://ornek.com → port 80; https://ornek.com → port 443). Nitekim Ünite 1'deki `curl -v` çıktısında, https:// kullandığınız için port 443'ü görmüştünüz — bu, sözleşmenin fiilen çalıştığının kanıtıydı. TLS/şifreleme detayına (yani HTTPS'in HTTP'den içerik düzeyinde farkına) bu hafta girilmez, bu Hf.12'nin konusudur; burada yalnız HTTP/HTTPS ayrımının ve port farkının farkında olmanız yeterli.

1.2.5 Ters vekil: isteği doğru yere yönlendiren kapı görevlisi

Büyük bir sitenin arkasında genelde tek bir uygulama sunucusu değil, bir **giriş noktası** vardır. İstemcinin isteği doğrudan uygulama sunucusuna değil, önce bu giriş noktasına — bir **ters vekile** — gider. Ters vekil (ör. nginx), gelen isteği inceleyip nereye yönlendireceğine karar verir.

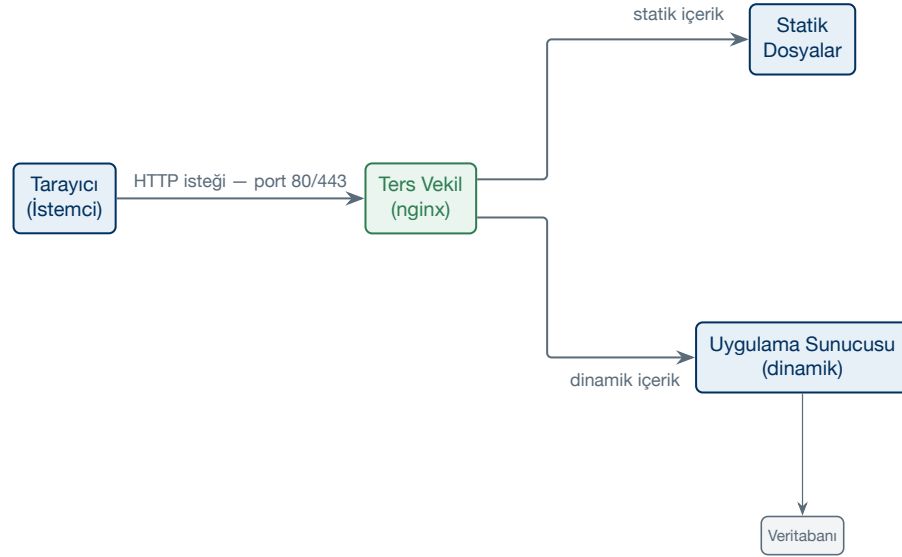
ters vekil:

istemciyle gerçek uygulama sunucusu arasına girip isteği yönlendiren aracı sunucu (*reverse proxy*).

Analoji

Ters vekili büyük bir binanın danışma/kapı görevlisine benzetin: ziyaretçi doğrudan ilgili ofise gitmez, önce danışmaya (ters vekile) gelir; danışma görevlisi ziyaretçiyi ya doğru ofise (dinamik uygulama sunucusuna) ya da arşive (statik dosyalara) yönlendirir. Ziyaretçi binanın iç yapısını bilmek zorunda değildir — tek bilmesi gereken, danışmaya gelmesidir.

Şekil 1.2 bu akışı gösterir: istemciden gelen istek önce ters vekile ulaşır (port 80/443 üzerinden); ters vekil isteği ya doğrudan diskteki statik dosyalara, ya da dinamik cevap üreten uygulama sunucusuna yönlendirir — uygulama sunucusu da gerektiğinde bir veritabanına başvurabilir.



Şekil 1.2: Standart web sunucu mimarisi: istemciden ters vekile, oradan statik dosyalara ya da dinamik uygulama sunucusuna dallanan istek akışı.

Bu haftada ters vekil yalnız **kavram düzeyinde** tanıtılır; Hf.14'te Docker Compose ile gerçek bir ters vekili fiilen kuracaksınız — o zaman React'in derlenmiş statik dosyalarını nginx'in doğrudan sunduğunu, /api/. . . yoluna gelen isteklerin ise FastAPI sunucunuza yönlendirildiğini göreceksiniz.

3 • Katılım Şimdi Siz Deneyin

`curl -I https://www.example.com` ile yalnız başlıkları isteyin (statik bir sayfa varsayımıyla). Şimdi aynı isteği birkaç saniye arayla iki kez tekrarlayın — cevap birebir aynı mı? Bunun dinamik bir uç noktadan (ör. güncel zaman döndüren bir test servisinden) farkı ne olurdu?

Geribildirim. Statik bir sayfanın başlıkları/gövdesi iki istekte de aynıdır — sunucu diskten aynı dosyayı gönderir, `Content-Length` gibi başlıklar da sabit kalır. Dinamik bir uç noktada ise her istek sunucuda **yeniden hesaplanan** bir cevap üretir; bu yüzden içerik (ör. bir zaman damgası ya da rastgele kimlik) her istekte değişebilir, hatta `Content-Length` bile farklı çıkabilir.

Kendinizi Sınavın 1.2

Bir ters vekilin (reverse proxy) istemciden gelen isteği doğrudan uygulama sunucusuna değil de önce kendisine almasının en az iki nedenini sıralayın.

5 • İzleme Aktarım — Başka Nerede?

Hf.14'te tam olarak bu ünitenin kavramını kuracaksınız: bir nginx ters vekili, React'in derlenmiş **statik** dosyalarını doğrudan sunacak, /api/. . . yoluna gelen istekleri ise **dinamik** cevap üreten FastAPI sunucusuna yönlendirecek — bu haftanın kavram olarak öğrendiği statik/dinamik ayrımı, o hafta gerçek bir `docker compose` yapılandırmasına dönüşecek.

1.3 WebApp Mimarisi: Üç Katman, API, SPA vs SSR

1 • Ön-öğretim Neden Önemli?

Bir bilet satış sistemi düşünün: kullanıcı bir koltuğa tıklar, sistem bu koltuğun hâlâ boş olup olmadığını kontrol eder, sonucu veritabanına yazar ve kullanıcıya “biletiniz alındı” der. Bu üç işi (gösterme, karar verme, kaydetme) tek bir dev fonksiyonda karıştırırsanız kod çabuk çözülmez bir yumağa dönüşür. Bu ünite, web uygulamalarının bu üç işi nasıl **ayrı katmanlara** böldüğünü ve bunun neden önemli olduğunu gösterir.

1 • Ön-öğretim Ön Bilgi

YG I/II’de Qt+SQLite ile yazdığınız masaüstü uygulamada arayüz kodu, iş mantığı ve veritabanı erişimi genelde aynı süreç, hatta çoğu zaman aynı fonksiyonlar içindeydi. Ünite 1-2’nin istemci-sunucu ve API’siz “düz” istek/cevap fikri de bu ünitenin temelidir — bu ünite, o düz istek/cevabın *sunucu tarafında* nasıl bölündüğünü gösterir.

Ünite 2, bir isteğin bir web sunucusuna nasıl ulaştığını gösterdi. Bu ünite, o sunucunun *içine* girip modern bir WebApp’in kendi iç mimarisini açar — ve bu dersin kullanacağı teknoloji yığını bu mimariye yerleştirir.

1.3.1 Üç katmanlı mimari

Modern bir web uygulaması, işini genelde üç ayrı sorumluluğa böler:

- **Sunum katmanı** (*presentation layer*): kullanıcının doğrudan gördüğü ve etkileşime girdiği arayüz. Koltuk seçim ekranı, “Satın Al” düğmesi — hepsi burada yaşar.
- **İş katmanı** (*business logic layer*): kuralların uygulandığı mantık. “Bir koltuk iki kez satılamaz” gibi bir kural tam olarak burada yaşar.
- **Veri katmanı** (*data layer*): bilginin kalıcı tutulduğu yer — hangi koltuğun satıldığı, hangi kullanıcının hangi bileti aldığı burada saklanır.

Üç katmanlı mimari: bir web uygulamasının işini sunum katmanı, iş katmanı ve veri katmanı olmak üzere üç ayrı sorumluluğa bölen tasarım (*three-tier architecture*).

Bu üçünü tek bir cümleyle tanımlayan terim **üç katmanlı mimari**dir. Web’de bu üçü ayrı süreçlerde, hatta ayrı makinelerde, ağ üzerinden konuşarak çalışabilir — YG I/II’nin Qt+SQLite masaüstü uygulamasında bu üçü aynı süreçteydi; web bunları fiziksel olarak da ayrıştırılabilir.

Analoji

Üç katmanlı mimariyi bir restoranın garson/mutfak/kiler düzenine benzetin: garson (sunum katmanı) siparişi alır ve yemeği müşteriye getirir, ama kendisi pişirmez; mutfak (iş katmanı) tarifi uygulayıp yemeği hazırlar, ama malzeme deposu değildir; kiler/depo (veri katmanı) malzemelerin saklandığı yerdir, ama kendisi yemek pişirmez. Her biri kendi işini yapar — garson pişirmeye kalkışmaz, mutfak malzeme depolamaya kalkışmaz. Bir sorun çıktığında (ör. yanlış yemek geldi) hangi katmanda arayacağınızı bilirsiniz: sipariş yanlış *iletildiyse* sorun garsonda, tarif yanlış *uygulandıysa* sorun mutfakta, malzeme *bozüksa*

sorun kilerdedir.

1.3.2 API: katmanlar arası sözleşme

Sunum katmanı ile iş katmanı ayrı süreçlerde çalışıyorsa, aralarında nasıl konuşacaklar? Bu soruyu yanıtlayan kavram **API**dir. Sunum katmanı, iş katmanından veri istemek ya da bir değişiklik yapmak istediğinde, önceden tanımlanmış bir sözleşmeye uyar: hangi isteğin hangi cevabı doğuracağı önceden bellidir.

API: bir yazılım bileşeninin bir başkasına önceden anlaşılmış bir sözleşmeyle sunduğu arayüz (*Application Programming Interface*).

Analoji

API'yi bir restoranın sipariş fişine benzetin: garson mutfağa doğrudan girip istediğini pişirmez; standart bir sipariş fişi doldurup mutfağa iletir. Fişin biçimi — hangi bilgi nereye yazılır, hangi alanlar zorunludur — önceden anlaşılmıştır. Mutfak fişi okur, siparişi hazırlar ve tabağı garsona geri verir. Garson mutfağın iç işleyişini (ocaklar nasıl dizilmiş, hangi tencere kullanılmış) bilmek zorunda değildir; yalnız fişin biçimini bilmesi yeterlidir — tıpkı bir API'nin istek/cevap biçiminin önceden tanımlı olması, ama arkasındaki uygulamanın gizli kalması gibi.

Bu ders boyunca API'leriniz **JSON** formatında veri taşıyacak — sunucu tarafı HTML şablonu üreten bir yaklaşım (Jinja2 gibi) bu derste hiç anlatılmaz; sunucunuz her zaman ham veriyi (JSON) döndürecek, o veriyi ekrana *nasıl* döküceğine sunum katmanı (React) karar verecektir.

1.3.3 SPA ile SSR arasındaki fark

Sunum katmanının sayfaları nasıl oluşturduğuna dair iki farklı yaklaşım vardır. **SPA**da tarayıcı bir kez HTML/JS indirir, sonrasında yalnızca API'den veri çekip sayfayı JavaScript ile günceller — React bu modeldedir. **SSR** ise bunun tam tersidir: *Server-Side Rendering* kısaltması olan bu modelde sunucu, her sayfa isteğinde baştan sona tam bir HTML sayfası üretilip gönderir.

SPA: tarayıcının tek seferde yüklediği HTML/JS ile veriyi API'den çekip arayüzü güncellediği model (*Single Page Application*).

Analoji

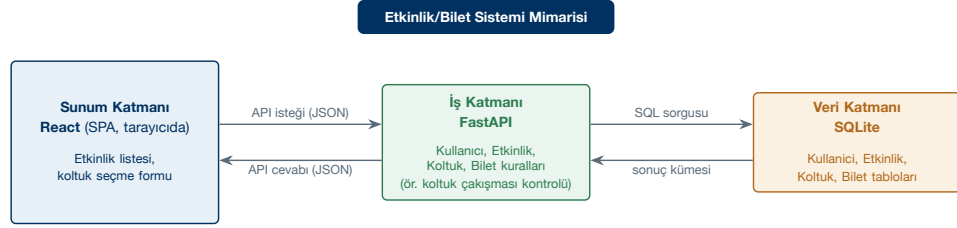
SPA ile SSR'ı bir kitapçıkla gezmek ile her soruda yeni broşür almaya benzetin: SPA bir kez tam bir kitapçık (uygulamanın kendisi) alır, sonra yalnızca güncel sayfaları (veriyi) API'den ister — kitapçığın kapağını, düzenini yeniden indirmenize gerek yoktur. SSR ise her soruda gişeden yeniden basılmış **komple** bir broşür (tam bir HTML sayfası) alır — her seferinde her şey baştan gelir.

Bu ders **SPA** yaklaşımını (React + Vite) izleyecek; SSR yalnız karşılaştırma amacıyla kavram düzeyinde tanıtılır, bu derste uygulanmaz.

1.3.4 Bu dersin yığınının şemaya yerleştirilmesi

Şimdi bu üç katmanı, dönem boyunca kullanacağınız somut teknolojilerle doldurun: **React** (Vite, TypeScript) tarayıcıda çalışan SPA'nızı oluşturarak sunum katmanına yerleşir; **FastAPI** (Python, Pydantic) API sağlayıcınız olarak iş katmanına yerleşir; **SQLite** (SQLAlchemy ORM üzerinden) verilerinizi kalıcı tutarak veri katmanına yerleşir.

Şekil 1.3, bu üç katmanlı Etkinlik/Bilet Sisteminizi somutlaştırır — bu şema, dönem boyunca defalarca geri döneceğiniz bir **harita**dır: Hf.4-5'te sunum katmanını (React bileşenleri), Hf.6'da iş katmanını (FastAPI uç noktaları), Hf.8'de veri katmanını (SQLAlchemy/SQLite) dolduracaksınız.



Şekil 1.3: Etkinlik/Bilet Sisteminin üç katmanlı mimarisi: React (sunum), FastAPI (iş), SQLite (veri) ve aralarındaki API/SQL değişimi.

İpucu

Bu şemaya her yeni hafta bir katman ya da bir bağlantı eklendiğinde, kendinize “şu an hangi katmandayım, hangi sözleşmeyi (API) kullanıyorum” diye sormak, koddaki bir parçanın *nereye ait olduğunu* netleştirir. Bu alışkanlık, büyüyen bir kod tabanında katmanların birbirine karışmasını önler.

3 • Katılım Şimdi Siz Deneyin

Şu üç cümleyi sunum/iş/veri katmanlarından birine yerleştirin: (a) “Koltuk seçim ekranında dolu koltuklar gri gösterilir.” (b) “Seçilen koltuğun hâlâ boş olduğu kontrol edilir.” (c) “Bilet kaydı kalıcı olarak saklanır.”

Geribildirim. (a) **sunum katmanı** — kullanıcıya görsel bir gösterim yapıyor, herhangi bir karar verilmiyor. (b) **iş katmanı** — burada bir kural/karar uygulanıyor (“hâlâ boş mu?”). (c) **veri katmanı** — kalıcı saklama işi burada. Üçü de ayrı sorumluluklardır ve web mimarisinde genelde ayrı bileşenlerde (React bileşeni, FastAPI uç noktası, SQLAlchemy modeli) yaşar.

Kendinizi Sınavın 1.3

Bilet sisteminde “aynı koltuğun iki kez satılmaması” kuralı üç katmandan hangisinde yer almalıdır ve neden?

5 • İzleme Aktarım — Başka Nerede?

Bu üç katmanlı şema (webapp-uc-katman), Etkinlik/Bilet Sisteminin dönem boyunca inşa edileceği haritadır: her yeni haftada bu şemanın bir parçasına dönüp “şu an hangi katmandayız?” diye sorabilirsiniz. Aynı üç katmanlı ayırım fikri, YG I/II’nin Kitaplık projesindeki **Kitap**, **Uye** sınıflarının verisiyle davranışının birlikte durmasından farklı bir şeydir — burada aynı ayırım (veri/davranış/gösterim) artık ağ üzerinden konuşan ayrı bileşenlere bölünmüştür.

1.4 HTTP İstek/Cevap Anatomisi

1 • Ön-öğretim 📣 Neden Önemli?

Tarayıcınızın Geliştirici Araçları'nın (DevTools) Ağ (Network) sekmesini ilk kez açtığınızda onlarca satır, başlık ve sayı karmaşık görünür. Oysa bunların hepsi aslında düz metin kurallarına uyan, okunması öğrenilebilir bir yapıdır. Bu ünite bu "karmaşık" görünümü, altındaki basit anatomiye indirger.

1 • Ön-öğretim 🔄 Ön Bilgi

Ünite 1-3'ün TCP bağlantısı, IP/port, üç katmanlı mimari fikrini hatırlayın. Ayrıca YG II'de soket üzerinden ham bayt gönderdiğiniz/ aldığınız deneyimi (*send/receive*) aklınızda tutun — HTTP de sonunda bu aynı TCP soket üzerinden gönderilen **düz metin**dir; bu ünite bunu size elle kanıtlayacak.

Önceki üç ünite ağın katmanlarını, sunucu mimarisini ve WebApp'in iç yapısını gösterdi. Şimdi bu tüm katmanların ortak konuştuğu dile, HTTP'nin isteğin ve cevabın kelime kelime nasıl yazıldığına bakma zamanı.

1.4.1 HTTP metotları: isteğin niyeti

HTTP metodu: bir HTTP isteğinin amacını belirten anahtar kelime (GET, POST vb.) (*HTTP method*).

Her HTTP isteği, ne yapmak istediğini bir **HTTP metodu** ile belirtir. En sık kullanılan dördü:

- **GET** — var olan bir veriyi **okur**, değiştirmez.
- **POST** — yeni bir veri/kaynak **oluşturur**.
- **PUT/PATCH** — var olan bir kaynağı **günceller**.
- **DELETE** — bir kaynağı **siler**.

Bu haftada bu dördünü yalnız isim ve niyet düzeyinde tanıyorsunuz; her birini gerçek bir API'de nasıl kullanacağınızı Hf.6'da derinlemesine göreceksiniz.

1.4.2 İstek anatomisi: satır satır

Bir HTTP isteği, aslında dört parçadan oluşan düz bir metindir: bir **istek satırı**, ardından **başlıklar**, ardından bir **boş satır**, ardından (varsa) bir **gövde**. Aşağıdaki, elle yazılmış ham bir örnek:

> Terminal

```
GET /etkinlikler HTTP/1.1
Host: biletsistemi.example.com
User-Agent: curl/8.4.0
Accept: application/json
```

Bu dört satırı sırayla açın. İlk satır (**GET /etkinlikler HTTP/1.1**) **istek satırıdır**: metot (**GET**), yol (**/etkinlikler**) ve HTTP versiyonu (**HTTP/1.1**) üç boşlukla ayrılmış tek satırda durur. Sonraki üç satır **başlıklardır** — her biri **Anahtar: Değer** biçiminde, isteğe dair ek bilgi taşır: **Host** hangi alan adına gidildiğini, **User-Agent**

başlık: istek/cevabın üst bilgi satırları (ör. içerik türü, sunucu adı) (*header*).

gövde:
istek/cevapta taşınan
asıl veri (varsa) (*body*).

isteği kimin (hangi programın) gönderdiğini, **Accept** istemcisinin hangi cevap biçimini kabul edeceğini söyler. Yukarıdaki örnekte gösterilmese de, başlıklardan sonra tam olarak bir **boş satır** gelir — bu boş satır, başlıkların bittiğini ve (varsa) gövdenin başladığını işaretler. **GET** isteklerinde genelde bir **gövde** yoktur — **GET** yalnızca var olan bir kaynağı istediği için ek veri taşımaya gerek kalmaz.

Şekil 1.4 bu dört parçayı, karşılık gelen cevap anatomisiyle yan yana gösterir.

1.4.3 Cevap anatomisi: satır satır

Bir HTTP cevabı da aynı dört parçalı yapıyı izler — yalnız ilk satır farklıdır. Aşağıdaki, yukarıdaki isteğe verilecek ham bir örnek cevap:

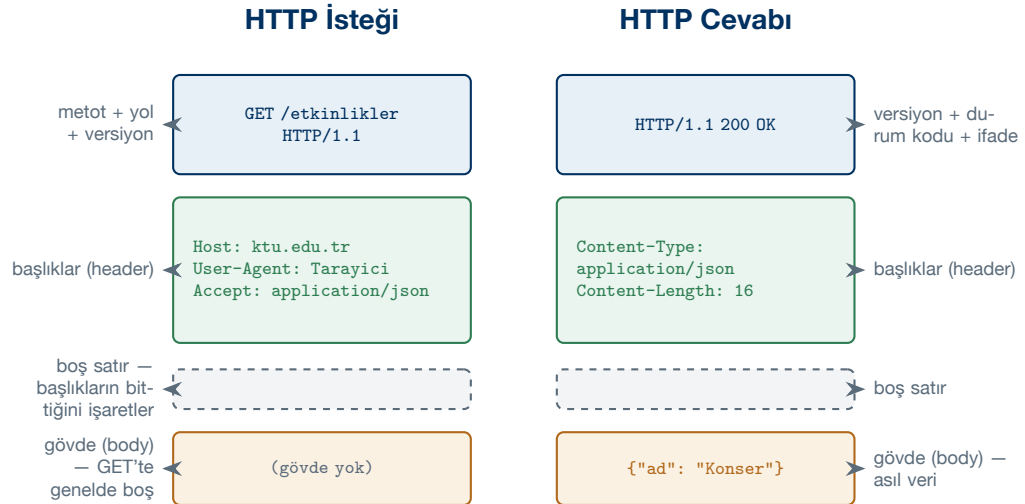
```
Beklenen Çıktı

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 52

{"etkinlik": "KTU Bahar Senligi", "kalanKoltuk": 42}
```

durum kodu:
sunucunun bir HTTP
cevabında isteğin
sonucunu bildiren üç
haneli kod (*status
code*).

İlk satır (**HTTP/1.1 200 OK**) **durum satırıdır**: HTTP versiyonu, bir **durum kodu** (**200**) ve kısa bir ifadeyle (**OK**) birlikte tek satırda durur. Sonraki iki satır yine başlıklardır: **Content-Type** gövdenin biçimini (burada JSON) bildirir, **Content-Length** gövdenin bayt cinsinden uzunluğunu söyler. Ardından bir **boş satır** gelir, ve son olarak **gövde**: sunucunun döndürdüğü asıl veri — bu örnekte kısa bir JSON nesnesi. **POST** gibi veri gönderen isteklerin aksine, bir **GET** cevabının gövdesi genelde doludur — çünkü cevabın tüm amacı istenen veriyi taşımaktır.



Şekil 1.4: Bir HTTP isteği ile cevabının anatomisi: istek/durum satırı, başlıklar, boş satır ve gövde.

Analoji

HTTP isteği/cevabı anatomisini bir mektup zarfına benzetin: zarfın üstündeki gönderen/alıcı adresi ve pul (başlıklar), zarfın içindeki mektup (gövde) — postacı zarfı hiç açmadan, üstündeki bilgiyle (başlıklarla) doğru yere yönlendirir; tıpkı bir sunucunun **Host** başlığına bakıp hangi siteye ait isteği işleyeceğine, ya

da bir tarayıcının **Content - Type** başlığına bakıp gövdeyi nasıl yorumlayacağına karar vermesi gibi.

1.4.4 Durum kodu aileleri

Durum kodları tek tek ezberlenecek yüzlerce sayı değildir; ilk rakamlarına göre ailelere ayrılırlar:

- **2xx** — başarılı (ör. **200 OK**);
- **3xx** — yönlendirme (istenilen kaynak başka bir yerde);
- **4xx** — istemci hatası (ör. **404 Bulunamadı** — isteği siz yanlış yaptınız);
- **5xx** — sunucu hatası (sunucu tarafında bir şey ters gitti).

Analoji

Durum kodunu gişe memurunun kısa cevabına benzetin: “Tamam, alın” (2xx), “Başka gişeye gidin” (3xx), “Böyle bir belge yok” (4xx), “Şu an sistemimiz arızalı” (5xx) — tek bir kısa kod, uzun bir açıklamanın yerini tutar. Bu haftada tüm kodları ezberlemenize gerek yok; yalnız hangi *ailenin* ne anlama geldiğini bilmeniz, bir hatanın kaynağının sizde mi (4xx) yoksa sunucuda mı (5xx) olduğunu ilk bakışta ayırt etmenizi sağlar.

1.4.5 curl -v ile canlı inceleme

Aşağıdaki kesit, gerçek bir **GET** isteğinin giden ve gelen tüm başlıklarını **curl -v** ile gösterir:

```
1 GET_CIKTI=$(curl -sS -v -m 10 --http1.1 "https://httpbin.org/get" 2>&1)
2 GET_DURUM=$?
3 if [ ${GET_DURUM} -ne 0 ]; then
4     echo "HATA: httpbin.org/get adresine baglanilamadi. Internet baglantinizi
      kontrol edin." >&2
5 else
6     echo "${GET_CIKTI}" | grep -E "^(> |< )"
7 fi
```

Çıktıda > ile başlayan satırlar sizin **giden isteğinizi** (istek satırı + başlıklar), < ile başlayan satırlar **sunucudan gelen cevabı** (durum satırı + başlıklar) gösterir — tam olarak bu ünitenin anlattığı iki anatomi, terminalde canlı görünür hâle gelir.

Şimdi gövdesi dolu bir **POST** isteğine bakın:

```
1 POST_CIKTI=$(curl -sS -m 10 -X POST "https://httpbin.org/post" \
2     -H "Content-Type: application/json" \
3     -d '{"ogrenci":"KTU","hafta":1}')
4 POST_DURUM=$?
5 if [ ${POST_DURUM} -ne 0 ]; then
6     echo "HATA: httpbin.org/post adresine baglanilamadi. Internet baglantinizi
      kontrol edin." >&2
7 else
8     echo "${POST_CIKTI}"
9 fi
```

-d ile verilen JSON, isteğin **gövdesinde** gönderilir; `httpbin.org` bu gövdeyi "json" alanında aynen geri yansıtır — `GET`'in aksine `POST`'un gövdesi doludur, çünkü `POST` sunucuya **yeni bir veri** taşımak için kullanılır.

İpucu

Aynı incelemeyi tarayıcınızda da yapabilirsiniz: Geliştirici Araçları'nı (DevTools) açıp Ağ (Network) sekmesine geçin, herhangi bir sayfayı yeniden yükleyin ve listedeki bir isteğe tıklayın — "Headers" sekmesinde tam olarak `curl -v`'nin gösterdiği istek satırı, başlıklar ve (varsa) gövde bilgisini, ama grafik bir arayüzde görürsünüz.

EKRAN GÖRÜNTÜSÜ YER TUTUCU

Bir web sayfası yeniden yüklenirken tarayıcının Geliştirici Araçları'nda (F12) Ağ (Network) sekmesi: istek listesi ve seçili bir isteğin Headers panelinde istek satırı, başlıklar ve durum kodu görünür olmalı.

`assets/ss/h01-devtools-network-sekmesi.png`

1.4.6 Ham soket ile HTTP: elle bir istek yazmak

HTTP'nin gerçekten "düz metin" olduğunu en iyi kanıtlayan yöntem, onu `curl`'e bırakmadan kendi elinizle bir soket üzerinden göndermektir. Aşağıdaki kesit, YG II'de kullandığınız `socket` mantığının Python karşılığıyla, bir HTTP `GET` isteğini elle bir dizge olarak kurup TCP bağlantısı üzerinden gönderir:

```
1 istek = (
2     f"GET / HTTP/1.1\r\n"
3     f"Host: {HOST}\r\n"
4     f"User-Agent: hafta01-soket-istemci\r\n"
5     f"Accept: text/html\r\n"
6     f"Connection: close\r\n"
7     f"\r\n"
8 )
```

Bu koda dikkatle bakın: `istek` değişkeni, Ünite 4'ün başında gördüğünüz dört parçalı istek metnini (`GET / HTTP/1.1` satırı + `Host/User-Agent/Accept` başlıkları + boş satır olarak `\r\n\r\n`) elle bir Python dizgesi olarak kurar.

Bu dizge hazır olduktan sonra sıra bağlantıyı açıp isteği göndermeye gelir:

```
1 with socket.create_connection((HOST, PORT), timeout=10) as soket:
2     soket.sendall(istek.encode("ascii"))
3     cevap = b""
4     while True:
5         parca = soket.recv(4096)
6         if not parca:
7             break
8         cevap += parca
9 print(cevap.decode("utf-8", errors="replace"))
```

`socket.create_connection` tam olarak YG II'de gördüğünüz `connectToHost` çağrısının Python karşılığıdır — yalnızca burada IP yerine `example.com` gibi bir alan adı verilir ve Python bunu içeride DNS ile çözer. `soket.sendall(...)` bu ham metni TCP bağlantısı üzerinden gönderir; `soket.recv(4096)` ise sunucudan gelen ham

baytları — durum satırından gövdeye kadar her şeyi — olduğu gibi okur. Sunucudan gelen cevap, tıpkı Ünite 4’ün başındaki örnek cevap gibi bir durum satırıyla başlar, başlıklarla devam eder, boş bir satırdan sonra gövdeyle biter — `curl -v`’nin sizin için otomatik yaptığı şeyi, burada siz elle yapmış oldunuz. Bağlantı hiç kurulamazsa oluşan `0SError` durumu, kaynak dosyanın sonundaki `except` bloğunda ele alınır; okuyucu bloğun tamamını `kod_ornekleri/hafta01/soket-istemci.py` dosyasında bulabilir.

Ortak Hata / Tuzak

Bu küçük istemcide `Connection: close` başlığının unutulması sık görülen bir hatadır: bu başlık olmadan bazı sunucular bağlantıyı açık tutmaya çalışır ve `soket.recv(4096)` döngüsü sunucunun bağlantıyı kapatmasını bekleyerek uzun süre (ya da zaman aşımına kadar) takılı kalabilir. `Connection: close` başlığı, sunucuya “cevabı gönderince bağlantıyı kapatabilirsin” demenin HTTP dilindeki yoludur.

3 • Katılım Şimdi Siz Deneyin

`curl -v https://httpbin.org/get` çalıştırın. Çıktıda `>` ile başlayan satırlar neyi, `<` ile başlayan satırlar neyi temsil eder? Gövde (body) hangi bölümde, nasıl ayırt edilir?

Geribildirim. `>` satırları **giden isteğinizi** (istek satırı + başlıklar), `<` satırları **gelen cevabı** (durum satırı + başlıklar) gösterir. Gövde, başlıklar bittikten sonra gelen **boş satırdan sonraki** kısımdır — `curl -v` çıktısında bu kısım genelde `>/<` işaretlerinden *arınmış* düz metin olarak görünür (JSON gövde burada, biçimlendirilmiş biçimde görünür).

Kendinizi Sınavın 1.4

Bir HTTP `GET` isteğinde gövde (body) genelde neden boştur, ama bir `POST` isteğinde neden dolu olabilir?

5 • İzleme Aktarım — Başka Nerede?

Bu haftanın kod planındaki `soket-istemci.py`, tam olarak bu ünitenin öğrettiği anatomiye (istek satırı + başlıklar + boş satır) elle bir dizgeye yazıp soket üzerinden gönderir — `curl`’ün sizin için otomatik yaptığını, siz burada elle yaptınız. Hf.6’da FastAPI ile bir uç nokta yazdığınızda, arka planda üretilen cevabın da bu aynı anatomiye (durum satırı + başlıklar + boş satır + gövde) uyduğunu hatırlayacaksınız — yalnızca bu kez o metni sizin yerinize FastAPI üretecek. Böylece bu haftanın öğrendiği dört katman — ağ, sunucu mimarisi, WebApp mimarisi, HTTP anatomisi — Etkinlik/Bilet Sisteminizin attığı her istekte birlikte devrede olacak.

Bölüm Özeti

Özet

Bu hafta web programlamaya girmeden önce, web'in üzerine oturduğu zemini katman katman gördünüz. İlk ünite, tarayıcınıza yazdığınız bir alan adının nasıl bir IP adresine ve porta dönüştüğünü öğrendiniz: IP adresi doğru *makineyi*, port o makinedeki doğru *servisi* bulur; DNS ise hatırlanabilir bir ismi bu IP adresine çeviren dağıtık bir rehberdir — YG II'de elle verdiğiniz IP'nin yerini gerçek web'de DNS almıştır. İkinci ünite bu bağlantının bir web sunucusunda nasıl karşılandığını gördünüz: istemci-sunucu modeli, statik içeriğin (her istekte aynı) ve dinamik içeriğin (her istekte yeniden üretilen) ayrımı, port 80/443 sözleşmesi ve isteği doğru yere yönlendiren bir ters vekilin rolü. Üçüncü ünite sunucunun *içine* girip modern bir WebApp'in üç katmanlı mimarisini — sunum, iş, veri — inceleyip bu dersin yığının (React, FastAPI, SQLite) bu şemaya yerleştirdiniz; SPA'nın API'den veri çeken tek-sayfa modelini SSR'ın her seferinde tam sayfa üreten modeliyle karşılaştırdınız. Son ünite tüm bu katmanların somut dilini, HTTP'yi, satır satır açtınız: bir isteğin (metot + yol + başlıklar + gövde) ve bir cevabın (durum satırı + başlıklar + gövde) anatomisini `curl -v` ile gözlemlediniz, hatta bu metnin aslında düz bir TCP soket üzerinden gönderildiğini kendi elinizle yazdığınız bir Python istemcisiyle kanıtladınız. Bu dört ünite birlikte, gelecek haftalarda kod yazarken sürekli geri döneceğiniz bir haritayı oluşturur: her yeni özellik, bu haritanın hangi katmanına, hangi HTTP değişimine karşılık geldiğini sorarak inşa edilecek. Gelecek hafta bu haritanın sunum katmanına ilk somut malzemeyi ekleyeceksiniz: statik bir Etkinlik/Bilet sayfasını HTML5 ve CSS3 ile bizzat yazacaksınız.

Kendinizi Sınavın Cevapları

Soru (Ünite 1 — IP adresi, port ve DNS birlikte ne iş yapar). Bir web sitesine tarayıcıdan eriştiğinizde IP adresi ile port numarası birlikte, birbirini tamamlayan iki farklı işi görür: IP adresi ağdaki doğru *makineyi* bulur (postane adresindeki bina gibi), port ise o makinede çalışan onlarca servisten hangisine gidileceğini belirler (aynı binadaki doğru daire gibi). Ama tarayıcı bir alan adına (`www.example.com`) baktığında elinde hazır bir IP adresi yoktur; bu ikiliyi kurabilmek için önce DNS'e başvurur — DNS, hatırlanabilir alan adını bu IP adresine çeviren dağıtık bir isim çözümleme sistemidir. Böylece tarayıcı, DNS'ten aldığı IP adresini ve protokolün varsayılan (ya da URL'de belirtilen) portunu kullanarak tam olarak `connect(ip, port)` çağırısına eşdeğer bir bağlantı kurar — YG II'de bu ikiliyi siz elinizle verirken, gerçek web'de DNS bu adımı sizin yerinize yapar.

Soru (Ünite 2 — ters vekilin araya girmesinin nedenleri). Bir ters vekilin istemciden gelen isteği doğrudan uygulama sunucusuna değil de önce kendisine almasının birden çok nedeni vardır. Birincisi, verimlilik: statik dosyaları (resim, CSS, derlenmiş HTML/JS) uygulama sunucusunu hiç meşgul etmeden, doğrudan diskten ya da önbellekten sunabilir — uygulama sunucusu yalnız gerçekten dinamik işlerle uğraşır. İkincisi, ölçeklenebilirlik: arka planda birden çok uygulama sunucusu çalışıyorsa ters vekil gelen trafiği bunlar arasında dağıtabilir (yük dengeleme), tek bir sunucu tıkanmaz. Üçüncüsü, tek bir giriş noktasından güvenlik ve TLS sonlandırma uygulayabilmesidir: HTTPS şifre çözme işi tek bir yerde toplanır, arkada-

ki uygulama sunucuları bu karmaşıklıkla uğraşmaz. Bu üç neden birlikte, gerçek dünyadaki web sunucularının neredeyse tamamının neden bir ters vekil önünde çalıştığını açıklar.

Soru (Ünite 3 — koltuk çakışması kuralı hangi katmanda durmalı). “Aynı koltuğun iki kez satılmaması” kuralı iş katmanında yer almalıdır. Bunun nedeni, bu kuralın bir iş kuralı olmasıdır — yani sistemin hangi durumları geçerli, hangilerini geçersiz saydığına dair bir karardır. Sunum katmanı yalnızca kullanıcıya koltukları gösterir ve tıklamayı iletir; kendi başına “bu koltuk hâlâ boş mu?” sorusuna karar veremez, çünkü güncel durumu bilmez ve tarayıcıda çalıştığı için kolayca atlatılabilir (kullanıcı isteği elle değiştirip kuralı baypas edebilir). Veri katmanı ise yalnızca kaydı saklar; “şu koltuk dolu” bilgisini tutar ama *ne zaman* dolu işaretleneceğine karar vermez. Kararı — isteği kontrol edip onaylamayı ya da reddetmeyi — iş katmanı verir, çünkü hem tüm istekleri gören hem de veri katmanına güvenilir biçimde erişebilen tek katman odur.

Soru (Ünite 4 — GET’in gövdesi neden boş, POST’unki neden dolu olabilir). Bir HTTP GET isteğinde gövde genelde boştur, çünkü GET’in amacı yalnızca var olan bir kaynağı istemektir — hangi kaynağın istendiği zaten istek satırındaki *yol* ve varsa sorgu parametreleriyle (*?anahtar=deger*) tam olarak belirtilmiştir; sunucuya ek bir veri göndermeye gerek yoktur. Buna karşılık bir POST isteği genelde sunucuda yeni bir kaynak oluşturmak ya da büyük bir değişiklik yapmak içindir — ör. yeni bir bilet kaydı oluşturmak; bu yeni kaynağın içeriği (hangi etkinlik, hangi koltuk, hangi kullanıcı) yalnızca URL’e sığmayacak kadar zengin bir veridir, bu yüzden bu veri isteğin gövdesinde, genelde JSON biçiminde taşınır. Kısacası: gövde, isteğin taşıdığı veri içindir; GET’in taşınacak verisi yoktur, POST’un genelde vardır.